

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 –DAST SIMULATION

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	K.Yalla Reddy	Reg. No.:	192465047
Section / Batch:	2024	Date:	12-08-2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I K.Yalla Reddy (192465047) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Rubrics:

Criteria	Weightage	Awarded Marks	Total Marks (100)
Understanding of Concepts	25%		
Experimental Design	30%		
Use of Tools	20%		
Analysis of Results	15%		
Documentation	10%		
Total per Question	100 Marks		

— 100

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA5802 - DEVSECOPS ENGINEERING AND APPLICATION SECURITY

CO3- ASSESSMENT TASK 1:

SIMULATE DYNAMIC APPLICATION SECURITY TESTING (DAST) AND SOFTWARE COMPOSITION ANALYSIS (SCA)

NAME: YALLA REDDY.K

REG.NO:192465047

1. Executive Summary

This report presents a realistic pre-release security assessment of a fictional e-commerce platform named SecureCart. The controlled test environment is modeled on OWASP Juice Shop, an intentionally insecure web application created for security training and security-tool testing. The current OWASP project remains actively maintained in 2026.

The assessment combines DAST, which examines the behavior of a running application, with SCA, which examines third-party and transitive dependencies. The educational simulation produces four DAST findings and four SCA findings. The combined risk score is 57, classified as HIGH.

2. Real-World Scenario

Scenario: Pre-release security assessment of an online shopping application. SecureCart provides product search, customer registration, login, shopping cart, checkout and REST APIs. Before release, the DevSecOps team deploys the application to an isolated staging environment and performs automated DAST and SCA checks.

For safe academic testing, the target is a controlled OWASP Juice Shop environment rather than a real production website. OWASP's Vulnerable Web Applications Directory describes Juice Shop as a deliberately vulnerable application for training and testing security tools.

3. Objectives

- Simulate DAST against a controlled e-commerce web application.
- Identify representative runtime web weaknesses.
- Simulate SCA for a Node.js dependency inventory.
- Classify findings by severity and calculate risk.
- Recommend remediation actions.
- Demonstrate DAST and SCA integration into DevSecOps CI/CD.

4. Tools and Technologies

- OWASP ZAP – DAST reference tool; its Baseline Scan spiders a target and performs passive scanning.
- npm audit – SCA reference tool for known dependency vulnerabilities.
- Python – educational simulation and JSON reporting.
- GitHub Actions – automated CI security workflow.

5. Architecture and Workflow

Developer Commit ☐ CI Pipeline ☐ Build/Test ☐ Staging Deployment ☐ DAST Scan ☐ SCA Scan ☐ Risk Aggregation ☐ Security Gate ☐ Remediation ☐ Re-scan ☐ Release

DAST tests the deployed application's external behavior. SCA examines the software supply chain. Together they provide complementary security coverage.

6. DAST Methodology

- Start the application in an isolated staging environment.
- Define target scope.
- Spider pages and API endpoints.
- Perform passive analysis.
- Perform active testing only on the authorized controlled target.
- Record evidence, severity and remediation.
- Re-scan after fixes.

7. DAST Simulation Results

ID	Finding	Severity	Area	Remediation
DAST-001	Content Security Policy header missing	MEDIUM	HTTP Security Headers	Define a restrictive Content-Security-Policy header.
DAST-002	Missing anti-CSRF protection	MEDIUM	Session / Request Protection	Utilize CSRF tokens or equivalent framework protection
DAST-003	Reflected XSS indicator	HIGH	Search/Input Handling	Use contextual output encoding and validate untrusted input
DAST-004	SQL injection indicator	HIGH	API Input Handling	Use parameterized queries and server-side validation

DAST summary: 4 findings — 2 HIGH and 2 MEDIUM. Risk score = 26, classified as HIGH.

8. DAST Finding Analysis

8.1 Missing Content Security Policy

A CSP restricts browser resource loading and can reduce the impact of some client-side injection weaknesses. Recommended action: define and test a restrictive policy.

8.2 Missing Anti-CSRF Protection

Sensitive state-changing requests should use CSRF protection appropriate to the authentication architecture. Recommended action: enable framework-supported CSRF protection or an equivalent design.

8.3 Reflected XSS Indicator

Untrusted input reflected into a response without contextual output encoding can become executable browser content. Recommended action: contextual output encoding, input validation and secure framework defaults.

8.4 SQL Injection Indicator

SQL injection can occur when untrusted input changes query structure. Recommended action: parameterized queries, prepared statements and server-side validation.

9. SCA Methodology

SCA identifies known vulnerabilities in direct and transitive third-party components. For a Node.js project, the dependency inventory is represented by package.json and package-lock.json. npm audit returns vulnerability information and remediation guidance.

- Collect package.json and package-lock.json.
- Build dependency tree.
- Run npm audit --json.
- Review severity, package and dependency path.
- Upgrade or replace vulnerable packages.
- Run tests and repeat the audit.

10. SCA Simulation Results

ID	Dependency	Severity	Issue	Remediation
SCA-001	transitive-package-A	CRITICAL	Known vulnerable dependency	Upgrade to a patched version or replace the package. ion.
SCA-002	transitive-package-B	HIGH	Known vulnerable dependency	Update the dependency chain to a fixed vers
SCA-003	direct-package-C	HIGH	Known vulnerable dependency	Upgrade the direct dependency and run regression tests.
SCA-004	transitive-package-D	MEDIUM	Known vulnerable dependency	Update the dependency or apply the vendor patch.

SCA summary: 1 CRITICAL, 2 HIGH and 1 MEDIUM. Risk score = 31, classified as HIGH.

11. Current Real-World Reference

A June 2026 OWASP CVE Lite case study measured an OWASP Juice Shop revision and reported 19 unique vulnerable packages: 3 critical, 10 high, 4 medium and 2 low, with 4 direct and 15 transitive vulnerable packages. This is included as a current real-world reference for the SCA portion; it is not claimed as the exact output of this educational simulation.

12. Risk Scoring

Scoring model: CRITICAL = 10, HIGH = 8, MEDIUM = 5, LOW =

2. DAST: 8 + 8 + 5 + 5 = 26 (HIGH).

SCA: 10 + 8 + 8 + 5 = 31 (HIGH).

Combined: 57 (**HIGH**).

13. DevSecOps Security Gate

A practical CI/CD gate can fail a build when critical vulnerabilities are detected or when high findings exceed an agreed threshold. npm audit supports severity thresholds through its audit-level option. The DAST stage should target only an authorized staging/test environment.

Recommended pipeline: Checkout ☐ Install ☐ SCA Audit ☐ Build/Test ☐ Staging ☐ ZAP DAST ☐ Report ☐ Security Gate ☐ Release.

14. Remediation Plan

- P1 – Fix critical/high dependency vulnerabilities.
- P1 – Fix SQL injection and XSS indicators.
- P1 – Add CSRF protection.
- P2 – Deploy and test CSP.
- P2 – Update transitive dependencies and regenerate lockfile.
- P3 – Automate DAST and SCA in CI/CD and track security trends.

15. Advantages

- DAST validates the running application's security posture.
- SCA identifies software supply-chain risk.
- The combined approach covers application and dependency layers.
- Automation makes testing repeatable.
- Severity gates help prioritize remediation.

16. Limitations

- The DAST and SCA findings are educational simulation results, not a production scan.
- Actual DAST results depend on authentication, crawler coverage and configuration.
- SCA results change as package versions and advisories change.
- Automated findings require validation and business-impact assessment.

17. Future Enhancements

- Run ZAP in Docker against a locally hosted Juice Shop staging instance.
- Add authenticated scanning for customer/admin workflows.
- Run npm audit --json against the exact package-lock.json.
- Generate SARIF reports.

- Add automated dependency update pull requests.
- Track vulnerability counts across builds.

18. Conclusion

This assessment demonstrates how DAST and SCA complement each other in DevSecOps. The e-commerce scenario provides a realistic business context, while the controlled OWASP Juice Shop environment provides a safe security-testing target. The DAST simulation found four representative web-layer issues and the SCA simulation found four dependency risks. The combined score of 57 is HIGH, showing why security gates should address both runtime exposure and software supply-chain risk before production release.

19. References

- OWASP Vulnerable Web Applications Directory – OWASP Juice Shop: <https://vwad.owasp.org/app/owasp-juice-shop/>
- OWASP Juice Shop v20.0.0, 13 May 2026: <https://owasp.org/blog/2026/05/13/juice-shop-v20>
- OWASP ZAP Baseline Scan: <https://www.zaproxy.org/docs/docker/baseline-scan/>
- npm audit documentation: <https://docs.npmjs.com/cli/v9/commands/npm-audit/>
- npm audit reports: <https://docs.npmjs.com/about-audit-reports/>
- OWASP CVE Lite Juice Shop case study: <https://owasp.org/cve-lite-cli/docs/case-studies/owasp-juice-shop/>

20. Appendix – Editable Project Files

- dast_simulation.py
- sca_simulation.py
- dast_results.json
- sca_results.json
- package.json
- .github/workflows/security.yml
- README.md
- REFERENCES.txt